Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/http/httpclient-guidelines
Exported: 2026-10-14. Source metadata date: 2025-10-22T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

Guidelines for using HttpClient

The System.Net.Http.HttpClient class sends HTTP requests and receives HTTP responses from a resource identified by a URI. An HttpClient instance is a collection of settings that's applied to all requests executed by that instance, and each instance uses its own connection pool, which isolates its requests from others. Starting in .NET Core 2.1, the SocketsHttpHandler class provides the implementation, making behavior consistent across all platforms.

DNS behavior

HttpClient only resolves DNS entries when a connection is created. It doesn't track any time to live (TTL) durations specified by the DNS server. If DNS entries change regularly, which can happen in some scenarios, the client won't respect those updates. To solve this issue, limit the lifetime of the connection by setting the PooledConnectionLifetime property, so that DNS lookup is repeated when the connection is replaced. Consider this example:

```
var handler = new SocketsHttpHandler
{
    PooledConnectionLifetime = TimeSpan.FromMinutes(15) // Recreate every 15 minutes
};
var sharedClient = new HttpClient(handler);
```

The preceding `HttpClient` is configured to reuse connections for 15 minutes. After the timespan specified by PooledConnectionLifetime has elapsed and the connection has completed its last associated request (if any), the connection is closed. If there are any requests waiting in the queue, a new connection is created as needed.

The 15-minute interval was chosen arbitrarily for illustration purposes. You should choose the value based on the expected frequency of DNS or other network changes.

Pooled connections

The connection pool for an HttpClient is linked to the underlying SocketsHttpHandler. Disposing an HttpClient disposes the pool only when the client owns and disposes the underlying handler. If a client uses a handler that it doesn't own, disposing the client doesn't affect the handler or its connections. IHttpClientFactory

creates clients with handlers that it manages, so it can reuse handlers and their connection pools. Sending another request to the same server using a different handler requires a new connection. As a result, there's a performance penalty for unnecessary connection creation. Moreover, TCP ports aren't released immediately after connection closure. (For more information on that, see TCP `TIME-WAIT` in RFC 9293.) If the rate of requests is high, the operating system limit of available ports might be exhausted. To avoid port exhaustion problems, reuse HttpClient instances for as many HTTP requests as possible.

Recommended use

To summarize recommended `HttpClient` use in terms of lifetime management, you should use either *long-lived* clients and set `PooledConnectionLifetime` (.NET Core and .NET 5+) or *short-lived* clients created by `IHttpClientFactory`.

- In .NET Core and .NET 5+:
  - Use a `static` or *singleton* HttpClient instance with PooledConnectionLifetime set to the desired interval, such as 2 minutes, depending on expected DNS changes. This solves both the port exhaustion and DNS changes problems without adding the overhead of IHttpClientFactory. If you need to be able to mock your handler, you can register it separately.

    > Tip
    >
    > If you only use a limited number of HttpClient instances, that's also an acceptable strategy. What matters is that they're not created and disposed with each request, as they each contain a connection pool. Using more than one instance is necessary for scenarios with multiple proxies or to separate cookie containers without completely disabling cookie handling.

  - Using IHttpClientFactory, you can have multiple, differently configured clients for different use cases. However, be aware that the factory-created clients are intended to be short-lived, and once the client is created, the factory no longer has control over it.

    The factory pools HttpMessageHandler instances, and, if its lifetime hasn't expired, a handler can be reused from the pool when the factory creates a new HttpClient instance. This reuse avoids any socket exhaustion issues.

    If you desire the configurability that IHttpClientFactory provides, we recommend using the typed-client approach.

- In .NET Framework, use IHttpClientFactory to manage your `HttpClient` instances. If you don't use the factory and instead create a new client instance for each request yourself, you can exhaust available ports.

  > Warning
  >
  > If your app requires cookies, it's recommended to avoid using IHttpClientFactory. Pooling the HttpMessageHandler instances results in sharing of CookieContainer objects. Unanticipated CookieContainer sharing might leak cookies between unrelated parts of the application. Moreover,

> when HandlerLifetime expires, the handler is recycled, meaning that all cookies stored in its CookieContainer are lost.

For more information about managing `HttpClient` lifetime with `IHttpClientFactory`, see `IHttpClientFactory` guidelines.

Resilience with static clients

It's possible to configure a `static` or *singleton* client to use any number of resilience pipelines using the following pattern:

```
using Microsoft.Extensions.Http.Resilience;
using Polly;

class MyClass
{
    static HttpClient? s_httpClient;

    MyClass()
    {
        var retryPipeline = new ResiliencePipelineBuilder<HttpResponseMessage>()
            .AddRetry(new HttpRetryStrategyOptions
            {
                BackoffType = DelayBackoffType.Exponential,
                MaxRetryAttempts = 3
            })
            .Build();

        var socketHandler = new SocketsHttpHandler
        {
            PooledConnectionLifetime = TimeSpan.FromMinutes(15)
        };
        var resilienceHandler = new ResilienceHandler(retryPipeline)
        {
            InnerHandler = socketHandler,
        };

        s_httpClient = new HttpClient(resilienceHandler);
    }
}
```

The preceding code:

- Relies on Microsoft.Extensions.Http.Resilience NuGet package.
- Specifies a transient HTTP error handler, configured with retry pipeline that with each attempt will exponentially backoff delay intervals.
- Defines a pooled connection lifetime of fifteen minutes for the `socketHandler`.
- Passes the `socketHandler` to the `resilienceHandler` with the retry logic.
- Instantiates a shared `HttpClient` given the `resilienceHandler`.

See also

- HTTP support in .NET

- HTTP client factory with .NET
- Make HTTP requests with the HttpClient
- Use IHttpClientFactory to implement resilient HTTP requests